☁️ Azure Deployment & System Integration Guide

📋 Table of Contents

☁️ Part 1: Azure Deployment Requirements

What You Need for Azure:

1. Azure Services Required

A. Compute (App Hosting)

```
Option 1: Azure App Service (Recommended for beginners)
├── Pricing Tier: B1 (Basic) - $13/month
├── Features: Auto-scaling, SSL, Custom domains
├── Supports: Python, Flask apps
└── Easy deployment via Git/VS Code

Option 2: Azure Virtual Machine
├── VM Size: B2s (2 vCPU, 4GB RAM) - $30/month
├── OS: Ubuntu 20.04 LTS
├── Features: Full control, can run ZAP
└── More complex but flexible

Option 3: Azure Container Instances
├── Container-based deployment
├── Cost: Pay per second of usage
├── Features: Isolated, scalable
└── Best for Docker containers
```

Recommendation: Start with **App Service** for easy deployment, move to **VM** if you need ZAP.

B. Database

```
Option 1: Azure SQL Database (Managed)
├── Tier: Basic (5 DTU) - $5/month
├── Storage: 2GB included
├── Features: Automatic backups, high availability
└── Better than SQLite for production

Option 2: Azure Database for PostgreSQL
├── Tier: Basic (1 vCore) - $25/month
├── Features: Open-source, powerful
└── Good for scaling

Option 3: Keep SQLite (Not recommended for production)
├── Cost: Free
├── Limitation: Not for multi-user
└── OK for demo/testing only
```

Recommendation: Azure SQL Database for small apps.

C. Storage

```
Azure Blob Storage
├── Purpose: Store generated reports (PDF, Excel, etc.)
├── Tier: Hot Access - $0.02/GB/month
├── Features: Unlimited storage, CDN integration
├── Access: Via URLs, APIs
└── Cost-effective for file storage

Why needed?
- Report files can be large
- Separate storage from app
- Easy to share/download
- Backup & archival
```

D. Additional Services (Optional)

```
Azure Key Vault
├── Purpose: Store API keys, secrets, connection strings
├── Cost: $0.03 per 10,000 operations
├── Security: Encrypted, access-controlled
└── Best Practice: Never hardcode secrets

Azure Monitor & Application Insights
├── Purpose: Logging, monitoring, alerts
├── Cost: First 5GB free/month
├── Features: Error tracking, performance monitoring
└── Essential for production

Azure Load Balancer (If scaling)
├── Purpose: Distribute traffic across instances
├── Cost: $18/month + data transfer
└── For high-traffic scenarios

Azure Content Delivery Network (CDN)
├── Purpose: Fast content delivery globally
├── Cost: $0.08/GB
└── For static files, reports
```

---

2. Changes Needed in Your Code

A. Configuration Changes

Current (Local):

python

```python
# config.py
DATABASE_CONFIG = {
    'db_name': 'scan_results.db'  # Local file
}

SERVER_CONFIG = {
    'host': '0.0.0.0',
    'port': 5000,
    'debug': True  # Development mode
}

ZAP_CONFIG = {
    'proxy_url': 'http://127.0.0.1:8080'  # Local ZAP
}
```

Azure (Production):

python

```python
# config.py
import os

DATABASE_CONFIG = {
    # Azure SQL Database
    'server': os.getenv('AZURE_SQL_SERVER'),
    'database': os.getenv('AZURE_SQL_DATABASE'),
    'username': os.getenv('AZURE_SQL_USERNAME'),
    'password': os.getenv('AZURE_SQL_PASSWORD'),
    'driver': '{ODBC Driver 17 for SQL Server}'
}

SERVER_CONFIG = {
    'host': '0.0.0.0',
    'port': int(os.getenv('PORT', 8000)),
    'debug': False  # Production mode
}

STORAGE_CONFIG = {
    # Azure Blob Storage for reports
    'connection_string': os.getenv('AZURE_STORAGE_CONNECTION'),
    'container_name': 'security-reports'
}

ZAP_CONFIG = {
    # ZAP on separate VM or container
    'proxy_url': os.getenv('ZAP_SERVICE_URL', 'http://zap-service:8080')
}
```

B. New Files Needed

1. requirements.txt (Updated)

txt

```
# Existing
flask>=2.3.0
python-owasp-zap-v2.4>=0.0.22
reportlab>=3.6.0
python-docx>=0.8.11
openpyxl>=3.0.9

# New for Azure
pyodbc>=4.0.35              # Azure SQL Database
azure-storage-blob>=12.19.0  # Blob Storage
azure-identity>=1.15.0       # Authentication
azure-keyvault-secrets>=4.7.0 # Key Vault
gunicorn>=21.2.0            # Production WSGI server
python-dotenv>=1.0.0        # Environment variables
```

2. .env (Environment Variables)

bash

```
# Azure SQL
AZURE_SQL_SERVER=yourserver.database.windows.net
AZURE_SQL_DATABASE=securityscanner
AZURE_SQL_USERNAME=admin
AZURE_SQL_PASSWORD=YourPassword123!

# Azure Storage
AZURE_STORAGE_CONNECTION=DefaultEndpointsProtocol=https;...

# ZAP Service
ZAP_SERVICE_URL=http://zap-vm-ip:8080

# Application
PORT=8000
FLASK_ENV=production
SECRET_KEY=your-secret-key-here
```

3. startup.sh (Azure Startup Script)

bash

```
#!/bin/bash
echo "Starting Security Scanner on Azure..."

# Install system dependencies
apt-get update
apt-get install -y unixodbc-dev

# Install Python packages
pip install -r requirements.txt

# Run database migrations (if any)
python init_db.py

# Start Gunicorn server
gunicorn --bind=0.0.0.0:8000 --workers=4 --timeout=300 app:app
```

4. Dockerfile (For containerization)

dockerfile

```
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    unixodbc-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY . .

# Expose port
EXPOSE 8000

# Run application
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "app:app"]
```

C. Code Modifications

1. Database Connection (Use Azure SQL instead of SQLite)

python

```python
# database.py (NEW FILE)
import pyodbc
import os

def get_db_connection():
    """Connect to Azure SQL Database"""
    server = os.getenv('AZURE_SQL_SERVER')
    database = os.getenv('AZURE_SQL_DATABASE')
    username = os.getenv('AZURE_SQL_USERNAME')
    password = os.getenv('AZURE_SQL_PASSWORD')

    connection_string = f"
        DRIVER={{ODBC Driver 17 for SQL Server}};
        SERVER={server};
        DATABASE={database};
        UID={username};
        PWD={password};
        Encrypt=yes;
        TrustServerCertificate=no;
        Connection Timeout=30;
    "

    return pyodbc.connect(connection_string)
```

2. Report Storage (Use Azure Blob instead of local files)

python

```python
# storage.py (NEW FILE)
from azure.storage.blob import BlobServiceClient
import os

def upload_report_to_blob(file_path, scan_id, format):
    """Upload report to Azure Blob Storage"""
    connection_string = os.getenv('AZURE_STORAGE_CONNECTION')
    container_name = 'security-reports'

    blob_service = BlobServiceClient.from_connection_string(connection_string)
    blob_client = blob_service.get_blob_client(
        container=container_name,
        blob=f'report_{scan_id}.{format}'
    )

    with open(file_path, 'rb') as data:
        blob_client.upload_blob(data, overwrite=True)

    # Return public URL
    return blob_client.url

def download_report_from_blob(scan_id, format):
    """Download report from Azure Blob Storage"""
    # Implementation here
    pass
```

---

3. ZAP Deployment on Azure

Problem: ZAP is a desktop application, needs special handling

Solutions:

Option 1: Separate VM for ZAP

```
Setup:
1. Create Ubuntu VM (Standard_B2s)
2. Install ZAP:
   - wget https://github.com/zaproxy/zaproxy/releases/download/v2.14.0/ZAP_2.14.0_Linux.tar.gz
   - tar -xvf ZAP_2.14.0_Linux.tar.gz
   - Run in daemon mode: ./zap.sh -daemon -host 0.0.0.0 -port 8080

3. Configure network:
   - Open port 8080
   - Whitelist your App Service IP
   - Set API key

4. Your app connects via:
   ZAP_SERVICE_URL=http://zap-vm-ip:8080
```

Option 2: ZAP in Docker Container

dockerfile

```
# zap-dockerfile
FROM owasp/zap2docker-stable

EXPOSE 8080

CMD ["zap.sh", "-daemon", "-host", "0.0.0.0", "-port", "8080", "-config", "api.key=changeme"]
```

Deploy to **Azure Container Instances**

Option 3: ZAP as a Service (Third-party)

```
Services like:
- StackHawk (Paid, managed ZAP)
- Probely (Security testing as a service)
- Detectify (Automated scanner)

Pros: No infrastructure management
Cons: Recurring cost, less control
```

Recommendation: Option 1 (Separate VM) for full control and cost-effectiveness.

---

4. Deployment Steps

Method 1: Azure App Service (Easy)

bash

```
# 1. Install Azure CLI
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

# 2. Login to Azure
az login

# 3. Create Resource Group
az group create --name SecurityScanner --location eastus

# 4. Create App Service Plan
az appservice plan create --name SecurityScannerPlan \
    --resource-group SecurityScanner \
    --sku B1 --is-linux

# 5. Create Web App
az webapp create --name my-security-scanner \
    --resource-group SecurityScanner \
    --plan SecurityScannerPlan \
    --runtime "PYTHON:3.11"

# 6. Configure App Settings (Environment Variables)
az webapp config appsettings set --name my-security-scanner \
    --resource-group SecurityScanner \
    --settings AZURE_SQL_SERVER="yourserver.database.windows.net"

# 7. Deploy Code
az webapp up --name my-security-scanner \
    --resource-group SecurityScanner \
    --runtime "PYTHON:3.11"
```

Method 2: Azure VM (More control)

bash

```
# 1. Create VM
az vm create --resource-group SecurityScanner \
    --name SecurityScannerVM \
    --image UbuntuLTS \
    --size Standard_B2s \
    --admin-username azureuser \
    --generate-ssh-keys

# 2. Open ports
az vm open-port --resource-group SecurityScanner \
    --name SecurityScannerVM --port 80
az vm open-port --resource-group SecurityScanner \
    --name SecurityScannerVM --port 443

# 3. SSH into VM
ssh azureuser@<VM-IP>

# 4. Install dependencies
sudo apt update
sudo apt install python3-pip nginx

# 5. Clone/Upload your code
# 6. Install requirements
pip3 install -r requirements.txt

# 7. Setup Nginx reverse proxy
# 8. Start application with systemd
```

---

5. Domain & SSL

```
1. Register Domain (optional):
  - GoDaddy, Namecheap, etc.
  - Cost: $10-15/year

2. Configure Custom Domain in Azure:
  az webapp config hostname add --webapp-name my-security-scanner \
      --resource-group SecurityScanner \
      --hostname www.yourdomain.com

3. Enable HTTPS (Free SSL):
  az webapp update --name my-security-scanner \
      --resource-group SecurityScanner \
      --https-only true
```

```
Azure provides free SSL certificate!
```

---

6. Monitoring & Maintenance

```
Setup Azure Monitor:
1. Application Insights for logging
2. Alerts for errors/downtime
3. Performance metrics
4. Cost tracking

Commands:
az monitor app-insights component create \
    --app my-security-scanner \
    --resource-group SecurityScanner \
    --location eastus
```

🏫 Part 2: University Management System Integration

What You Need:

1. Authentication & Authorization

University System has users:

```
- Students
- Faculty
- Admin
- Security Officers
```

Your scanner needs to:

- Authenticate users via university SSO
- Check permissions (who can scan what)
- Limit scans per user
- Track who performed scans

Integration Requirements:

A. Single Sign-On (SSO)

```
University likely uses:
├── LDAP (Lightweight Directory Access Protocol)
├── Active Directory (Microsoft AD)
├── SAML 2.0 (Security Assertion Markup Language)
└── OAuth 2.0 / OpenID Connect

Your app needs:
├── python-ldap (for LDAP)
├── Flask-SAML (for SAML)
├── authlib (for OAuth/OIDC)
└── Flask-Login (session management)
```

Example Flow:

```
1. Student clicks "Login"
2. Redirected to university SSO
3. Student enters university credentials
4. SSO validates and sends token
5. Your app verifies token
6. Student logged in, can scan
```

B. Role-Based Access Control (RBAC)

Define Roles:

```python
```

```
ROLES = {
    'student': {
        'can_scan': True,
        'scan_limit': 5,  # per day
        'max_scan_time': 10,  # minutes (quick scan only)
        'can_download_reports': True,
        'formats_allowed': ['html', 'pdf']
    },
    'faculty': {
        'can_scan': True,
        'scan_limit': 20,
        'max_scan_time': 60,  # standard scan
        'can_download_reports': True,
        'formats_allowed': ['html', 'pdf', 'excel', 'word']
    },
    'security_officer': {
        'can_scan': True,
        'scan_limit': None,  # unlimited
        'max_scan_time': None,  # any scan type
        'can_download_reports': True,
        'formats_allowed': 'all',
        'can_view_all_scans': True,
        'can_delete_scans': True
    },
    'admin': {
        'all_permissions': True
    }
}
```

---

2. APIs to Integrate

A. University APIs You'll Need Access To:

```
1. Authentication API
   GET /auth/login
   POST /auth/validate-token

2. User Directory API
   GET /users/{id}
   GET /users/by-email/{email}

3. Department API (optional)
   GET /departments
   GET /departments/{id}/users

4. Course Management API (if scanning course websites)
   GET /courses
   GET /courses/{id}/website-url
```

Example Integration:

```python
# Authenticate with university
import requests

def verify_university_user(token):
    """Verify user with university API"""
    response = requests.post(
        'https://university.edu/api/auth/validate',
        headers={'Authorization': f'Bearer {token}'}
    )

    if response.status_code == 200:
        user_data = response.json()
        return {
            'id': user_data['user_id'],
            'name': user_data['name'],
            'email': user_data['email'],
            'role': user_data['role'],
            'department': user_data['department']
        }
    return None
```

B. APIs You'll Provide to University:

```
1. Scan Management API
   POST /api/v1/scan/create
   GET /api/v1/scan/{id}/status
   GET /api/v1/scan/{id}/results

2. Report API
   GET /api/v1/report/{id}/{format}

3. Statistics API
   GET /api/v1/stats/department/{dept_id}
   GET /api/v1/stats/user/{user_id}

4. Webhook for notifications
   POST /api/v1/webhook/scan-complete
```

---

3. Database Schema Changes

Add New Tables:

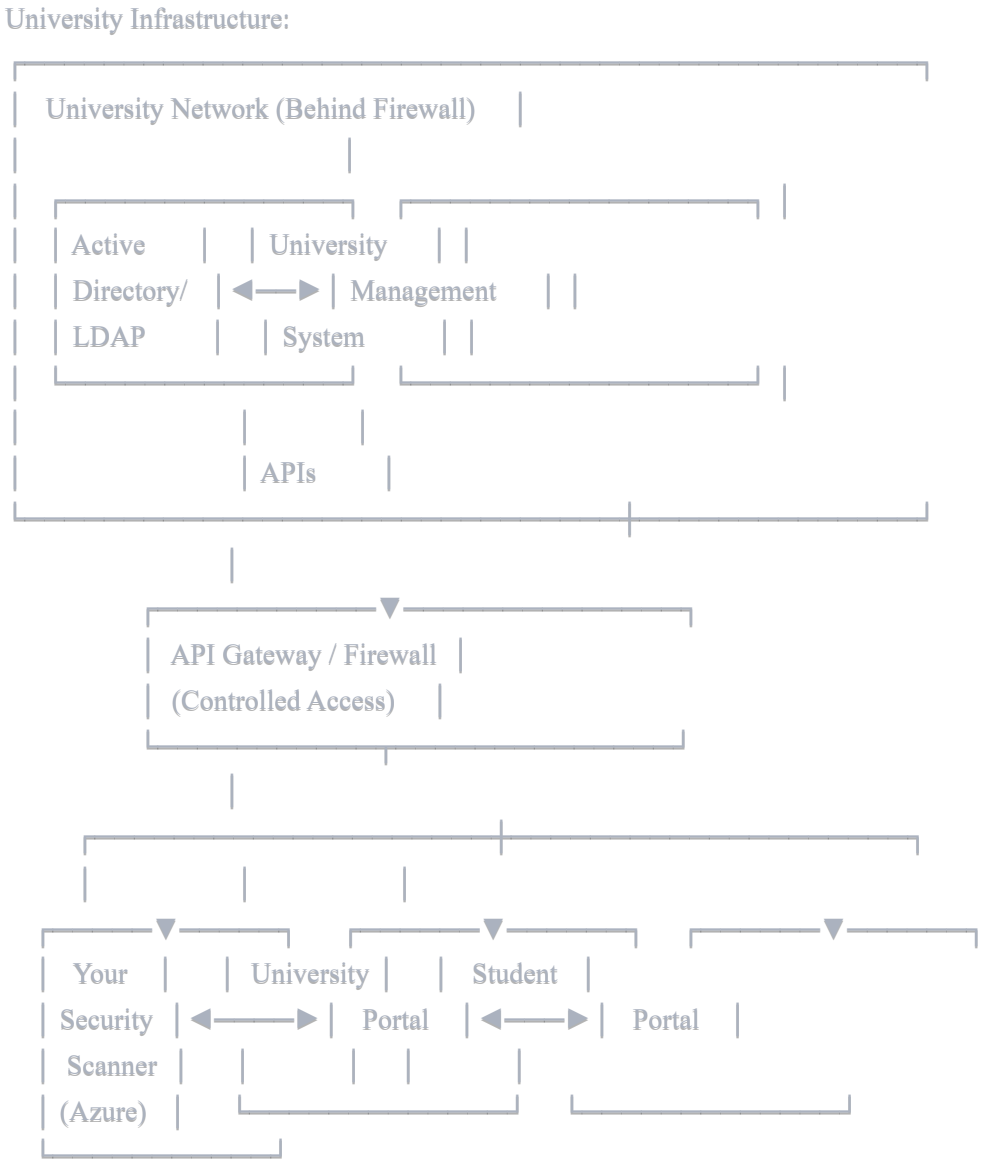
sql

```sql
-- Users table (sync with university)
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    university_id VARCHAR(50) UNIQUE,
    email VARCHAR(255),
    name VARCHAR(255),
    role VARCHAR(50),
    department VARCHAR(100),
    scan_quota INTEGER,
    created_at TIMESTAMP
);

-- Departments table
CREATE TABLE departments (
    id INTEGER PRIMARY KEY,
    name VARCHAR(255),
    code VARCHAR(50),
    scan_limit INTEGER
);

-- Scan permissions
CREATE TABLE scan_permissions (
    id INTEGER PRIMARY KEY,
    user_id INTEGER,
    target_url VARCHAR(500),
    permission_type VARCHAR(50),  -- 'own', 'department', 'all'
    granted_by INTEGER,
    granted_at TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Audit logs
CREATE TABLE audit_logs (
    id INTEGER PRIMARY KEY,
    user_id INTEGER,
    action VARCHAR(100),
    details TEXT,
    ip_address VARCHAR(45),
    timestamp TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Update scans table
ALTER TABLE scans ADD COLUMN user_id INTEGER;
```

```
ALTER TABLE scans ADD COLUMN department_id INTEGER;
ALTER TABLE scans ADD COLUMN scan_purpose VARCHAR(255);
```

---

4. Integration Architecture



---

5. Specific Integration Scenarios

Scenario A: Scanning Department Websites

Workflow:
1. Faculty logs in via university SSO
2. Selects "Scan Department Website"
3. System fetches department URL from university API
4. Verifies faculty has permission
5. Initiates scan
6. Results stored with department tag
7. Department admin receives notification

Scenario B: Student Project Scanning

Workflow:
1. Student submits project URL in course portal
2. Course portal calls your API:

```
POST /api/v1/scan/create
{
  "url": "student-project.com",
  "requester": "student_id",
  "course": "CS401",
  "scan_type": "quick"
}
```

3. Your scanner performs scan
4. Results sent back via webhook
5. Course portal displays security grade

Scenario C: Compliance Reporting

Workflow:
1. Security officer logs in
2. Views dashboard of all university sites
3. Filters by department/date
4. Generates compliance report
5. Shows trends over time
6. Exports to Excel for management

---

6. Network & Security Considerations

University Requirements:
├── IP Whitelisting (Your Azure IP must be allowed)
├── VPN Connection (For accessing internal systems)
├── Firewall Rules (Specific ports only)
├── SSL/TLS (Encrypted communication)

```
└── API Rate Limiting (Respectful usage)

Your Requirements:
├── Secure API endpoints (HTTPS only)
├── API authentication (API keys/OAuth)
├── Input validation (Prevent injection)
├── Rate limiting (Prevent abuse)
├── Audit logging (Track all actions)
└── Data encryption (At rest and in transit)
```

---

7. Compliance & Legal

```
University Will Ask For:
├── Data Privacy Policy (FERPA compliance for US universities)
├── Security Audit Report
├── Penetration Testing Results
├── Uptime SLA (Service Level Agreement)
├── Data Retention Policy
├── Incident Response Plan
└── Insurance Certificate (For liability)
```

---

8. Communication Channels

A. Email Notifications

python

```python
# Using Azure Communication Services
from azure.communication.email import EmailClient

def send_scan_complete_email(user_email, scan_id):
    client = EmailClient.from_connection_string(connection_string)

    message = {
        "content": {
            "subject": "Security Scan Complete",
            "plainText": f"Your scan #{scan_id} is complete.",
            "html": "<html>...</html>"
        },
        "recipients": {
            "to": [{"email": user_email}]
        }
    }

    client.send(message)
```

B. Webhooks

```python
# Notify university system when scan completes
def notify_university_webhook(scan_id, results):
    webhook_url = 'https://university.edu/api/webhooks/scan-complete'

    payload = {
        'scan_id': scan_id,
        'status': 'completed',
        'summary': {
            'total': results['total'],
            'high': results['high'],
            'medium': results['medium'],
            'low': results['low']
        },
        'report_url': f'https://yourapp.azurewebsites.net/report/{scan_id}'
    }

    requests.post(webhook_url, json=payload, headers={'X-API-Key': api_key})
```

C. Dashboard Integration

```html
```

```
<!-- Embed in university portal -->
<iframe src="https://yourapp.azurewebsites.net/embed/dashboard?token={auth_token}"
        width="100%" height="600px"
        frameborder="0"></iframe>
```

🏢 Part 3: Other Systems Integration

A. Integration with Hospital Management System

Use Cases:
- Scan patient portal for vulnerabilities
- Scan appointment booking system
- Scan medical records access (HIPAA compliant)

Special Requirements:
- HIPAA compliance
- PHI (Protected Health Information) handling
- Extra security measures
- Audit trails

B. Integration with E-commerce Platform

Use Cases:
- Scan product pages
- Test payment gateway security
- Check PCI DSS compliance

Special Requirements:
- PCI DSS compliance
- Payment data handling
- Shopping cart security
- API testing

C. Integration with Government Portal

Use Cases:
- Scan citizen services portals
- Check security of public websites
- Compliance with government standards

Special Requirements:
- Government security clearance

- On-premise deployment option
- Air-gapped network support
- NIST framework compliance

💰 Part 4: Cost Estimation

Azure Deployment Costs (Monthly):

```
Basic Setup:
├── App Service (B1): $13/month
├── Azure SQL Database (Basic): $5/month
├── Blob Storage (10GB): $0.20/month
├── VM for ZAP (B2s): $30/month
├── Bandwidth (100GB): $8/month
└── Total: ~$56/month

Medium Scale:
├── App Service (S1): $70/month
├── Azure SQL (S0): $15/month
├── Blob Storage (50GB): $1/month
├── VM for ZAP (B4ms): $120/month
├── Application Insights: $5/month
├── Bandwidth (500GB): $40/month
└── Total: ~$251/month

Enterprise:
├── App Service (P1v2): $146/month
├── Azure SQL (S3): $100/month
├── Blob Storage (500GB): $10/month
├── Multiple ZAP VMs: $300/month
├── Load Balancer: $18/month
├── Monitor/Insights: $50/month
├── Bandwidth (2TB): $160/month
└── Total: ~$784/month
```

University Integration Costs:

```
One-time:
├── Integration Development: $5,000-$10,000
├── Security Audit: $2,000-$5,000
├── Legal/Compliance Review: $1,000-$3,000
└── Total: $8,000-$18,000
```

```
Recurring:
├── Azure Hosting: $56-$251/month
├── Domain + SSL: $15/month
├── Support & Maintenance: $500-$1,000/month
└── Total: $571-$1,266/month
```

✅ Summary Checklist

For Azure Deployment:

- [ ] Azure account with active subscription
- [ ] Code modifications for cloud
- [ ] Database migration to Azure SQL
- [ ] File storage to Azure Blob
- [ ] ZAP deployment strategy
- [ ] SSL certificate
- [ ] Monitoring setup
- [ ] Backup strategy

For University Integration:

- [ ] SSO integration (LDAP/SAML/OAuth)
- [ ] User role management
- [ ] API access to university systems
- [ ] Network access (VPN/Whitelist)
- [ ] Legal agreements signed
- [ ] Compliance certifications
- [ ] Training materials
- [ ] Support documentation

Key Takeaway:

- Azure deployment is **straightforward** but needs code modifications
- University integration needs **APIs, authentication, and permissions**
- Costs range from **$56/month (basic)** to **$784/month (enterprise)**
- Integration with university is more about **permissions and policies** than technology

Bhai koi specific part ka detail chahiye? 🚀